3º Entregable BBDD - Scripts SQL | CS2-SVSS - Jose Angel Alejo Sillero

INFORMACIÓN DEL DOCUMENTO

Campo	Valor
Autor	José Ángel Alejo Sillero
Institución	Instituto Tecnológico Pablo de la Torre
Módulo	Base de Datos - 1º Desarrollo de Aplicaciones Web (DAW)
Proyecto	CS2-SVSS - Counter Strike 2 Stats Visualizing System (Fragify)
Tipo de Entregable	Entregable Unidad 3 - Scripts SQL y Diseño de Base de Datos
Fecha de creación	23 de enero de 2026
Última edición	23 de enero de 2026
Versión	1.0 (Documento Final)
Archivo SQL entregado	jose_angel_alejo_sillero.sql
Descripción	Documentación completa del proceso de desarrollo de scripts SQL, incluyendo decisiones técnicas, justificaciones de diseño basadas en la ERS (Entregable 1) y Modelo ER/LR (Entregable 2), e implementación de características avanzadas como vistas estratégicas y control de acceso basado en roles (RBAC).

CONTROL DE VERSIONES

VERSIÓN	FECHA	CAMBIOS	AUTOR
1.0	**23/01/2026**	**Documento final con todas las secciones completas, vistas, roles y justificaciones**	**José Ángel Alejo Sillero**

TABLA DE CONTENIDOS

- 3º Entregable BBDD - Scripts SQL | CS2-SVSS - Jose Angel Alejo Sillero
  - 1. INTRODUCCIÓN
  - 2. ANÁLISIS DE REQUISITOS
    - 2.1 Fuentes de Referencia
    - 2.2 Requisitos Clave Identificados
  - 3. DISEÑO DE TABLAS
    - Tabla 1: JUGADORES_CS2
      - Justificación de PK Natural vs Surrogate:
    - Tabla 2: SERVIDORES_CS2
    - Tabla 3: ESTADISTICAS_CS2
    - Tabla 4: RANKINGS_CS2
    - Tabla 5: PARTIDAS_CS2
    - Tabla 6: REPORTES_CONDUCTA
    - Tabla 7: LOGS_AUDITORIA
  - 4. RESTRICCIONES DE INTEGRIDAD DE DATOS
    - 4.1 JUGADORES_CS2
    - SERVIDORES_CS2
    - 4.2 ESTADISTICAS_CS2
    - 4.3 RANKINGS_CS2
  - 5. ÍNDICES Y OPTIMIZACIÓN DE RENDIMIENTO
    - 5.1 Estrategia de Indexación
    - 5.2 Índices por Tabla
  - 6. VISTAS ESTRATÉGICAS (VIEWS)
    - 6.1 Vista 1: vw_ranking_jugadores_elo
    - 6.2 Vista 2: vw_estadisticas_jugador_resumen
    - 6.3 Vista 3: vw_reportes_pendientes_valve
    - 6.4 Vista 4: vw_partidas_recientes_jugador
    - 6.5 Vista 5: vw_rendimiento_por_mapa
    - 6.6 Justificación de Ausencia de Vistas Adicionales
  - 7. PROCESO DE CREACIÓN
    - 7.1 Metodología
    - 7.2 Etapas Ejecutadas
      - 7.2.1 Etapa 1: Creación de estructura base
      - 7.2.2 Etapa 2: Creación de 7 tablas operacionales
      - 7.2.3 Etapa 3: Implementación de restricciones
      - 7.2.4 Etapa 4: Indexación estratégica
      - 7.2.5 Etapa 5: Vistas optimizadas
      - 7.2.6 Etapa 6: Control de acceso
  - 8. DECISIONES TÉCNICAS
    - 8.1 Motor de Base de Datos y Configuración
      - MySQL 8.0.11 + InnoDB
    - 8.2 Juego de Caracteres
      - utf8mb4 (UTF-8 de 4 bytes)
    - 8.3 Formas Normales
      - 1FN, 2FN, 3FN implementadas

        - 8.4 Claves Primarias: Natural vs Surrogate
            - JUGADORES_CS2: PK Natural (steam_id64)
            - SERVIDORES_CS2, ESTADISTICAS_CS2, etc.: PK Surrogate (AUTO_INCREMENT INT)
        - 8.5 Restricciones de Integridad
            - CHECK Constraints
        - 8.6 Foreign Keys
        - 8.7 Índices
        - 8.8 Vistas (Views)
            - 5 vistas estratégicas
        - 8.9 Roles y Control de Acceso
            - RBAC (Role-Based Access Control)
        - 8.10 Charset y Collation
            - utf8mb4_unicode_ci (case-insensitive, Unicode Collation Algorithm)
    - 9. ROLES, USUARIOS Y PERMISOS
        - 9.1 Rol 1: ADMINISTRADOR DEL SISTEMA (admin_cs2)
        - 9.2 Rol 2: SERVICIO API (api_service)
        - 9.3 Rol 3: INSPECTOR DE VALVE (valve_inspector)
        - 9.4 Rol 4: JUGADOR FRAGIFY (jugador_fragify) - 9.5 Rol 5: PROFESOR/EVALUADOR (profesor_evaluador) - 10. CONCLUSIÓN
        - 10.1 Cumplimiento de Requisitos
        - 10.2 Estadísticas de Entrega
        - 10.3 Beneficios y Fortalezas del Diseño
        - 10.4 Próximos Pasos Recomendados
    - DECLARACIÓN DE AUTORÍA

---

1. INTRODUCCIÓN

Este documento constituye la **memoria técnica del Entregable Tema 3**, que comprende el **desarrollo completo de la base de datos SQL** para el proyecto **CS2-SVSS (Fragify)**. Basándose en la **ERS (Especificación de Requisitos de Software)** del **Entregable 1** y en el **Modelo Entidad-Relación (ER) y Lógico-Relacional (LR)** del **Entregable 2**, se ha diseñado e implementado de forma integral:

- **7 tablas normalizadas** (1FN, 2FN, 3FN) con **integridad referencial** completa.
- **10 restricciones CHECK** validando **consistencia de datos** a nivel de BD.
- **13 índices estratégicos** optimizando **rendimiento y velocidad de consultas**.
- **5 vistas (Views)** para **precálculos de consultas complejas frecuentes**.
- **4 roles con permisos diferenciados** implementando **RBAC (Role-Based Access Control)**.

El objetivo de este entregable es **proporcionar una base de datos enterprise-grade**, robusta, segura y escalable que cumpla con los **principios ACID**, los **estándares IEEE 830-1998**, y prepare el ecosistema técnico para futuras expansiones del sistema (frontend web, integración con APIs de Valve, etc.). Este documento actúa como **guía de implementación** y **justificación técnica** para los administradores de bases de datos y equipo de desarrollo que mantenga el sistema CS2-SVSS.

---

2. ANÁLISIS DE REQUISITOS

2.1 Fuentes de Referencia

- **ERS CS2-SVSS_Jose-Angel-Alejo.md** (Entregable 1): Define entidades, atributos, relaciones y requisitos funcionales de almacenamiento, integridad, seguridad y mantenimiento.
- **Modelado_CS2SVSS_JAAlejo.md** (Entregable 2): Transforma el modelo conceptual (ER) al relacional (LR), aplicando formas normales y reglas de integridad.

2.2 Requisitos Clave Identificados

Aspecto	Requisito	Cumplimiento
Almacenamiento	Datos de jugadores, estadísticas, rankings, partidas, servidores, reportes y logs	7 tablas
Integridad	Claves primarias, FKs con reglas de cascada, CHECK constraints	10 restricciones
Seguridad	Control de acceso basado en roles, permisos diferenciados	4 roles
Rendimiento	<1.5s por consulta típica, 90% consultas exitosas	13 índices + 5 vistas
Normalización	1FN, 2FN, 3FN	Verificado
Compatibilidad	MySQL 8.0.11, InnoDB, utf8mb4	Implementado

3. DISEÑO DE TABLAS

Tabla 1: JUGADORES_CS2

Propósito: Almacenar información básica de jugadores registrados en Counter-Strike 2.

Clave Primaria: `steam_id64` (VARCHAR(17)) - PK Natural

Justificación de PK Natural vs Surrogate:

- Se eligió Steam ID64 como clave primaria natural porque:
    1. Es un identificador único e invariable de Valve (nunca cambia)
    2. Reduce redundancia en relaciones (no necesitamos campo adicional)
    3. Facilita sincronización con APIs de Valve
    4. Alternativa viable (usar surrogate INT) sería innecesariamente redundante

Restricciones:

- `chk_steam_id64_format`: Valida formato exacto (17 dígitos numéricos)
- `chk_fecha_registro`: Evita registros con fechas futuras
- `chk_ultima_actualizacion`: Detecta desincronizaciones de APIs

Índices:

- `idx_nombre_usuario`: Búsquedas por nombre en frontend
- `idx_estado_verificacion`: Filtrado de cuentas verificadas
- `idx_estado_actividad`: Análisis de jugadores activos/inactivos

Tabla 2: SERVIDORES_CS2

Propósito: Información de servidores de juego donde se alojan partidas.

Clave Primaria: `id_servidor` (INT AUTO_INCREMENT) - PK Surrogate

Justificación de PK Surrogate:

- No hay identificador natural estable de Valve para servidores
- Múltiples atributos (IP, región) pueden cambiar
- Surrogate INT es más eficiente en FKs que VARCHAR(45)

Relación: 1:N con PARTIDAS_CS2

Tabla 3: ESTADISTICAS_CS2

Propósito: Estadísticas agregadas de rendimiento por jugador.

Clave Primaria: `id_estadistica` (INT AUTO_INCREMENT) - PK Surrogate

Relación: 1:1 lógico con JUGADORES_CS2 (aunque 1:N en BD para posibles históricas)

FK: `steam_id64` → `jugadores_cs2(steam_id64)` con ON DELETE CASCADE

Justificación de ON DELETE CASCADE:

- Si un jugador es eliminado, sus estadísticas deben eliminarse
- Mantiene integridad referencial
- Cumple con RGPD (derecho al olvido)

Tabla 4: RANKINGS_CS2

Propósito: Rankings competitivos PREMIERE y por mapa.

Clave Primaria: `id_ranking` (INT AUTO_INCREMENT) - PK Surrogate

Campo especial: `tipo_ranking` (ENUM) permite diferenciar PREMIERE vs MAPA

Justificación de ENUM:

- Restringe valores válidos a nivel de BD
- Mejora eficiencia de almacenamiento
- Facilita consultas con WHERE tipo_ranking = 'PREMIERE'

Tabla 5: PARTIDAS_CS2

Propósito: Detalles de cada partida jugada.

Clave Primaria: `id_partida` (VARCHAR(50)) - PK Natural (de Valve)

FKs:

- `steam_id64` → `jugadores_cs2` (ON DELETE CASCADE)
- `id_servidor` → `servidores_cs2` (ON DELETE SET NULL)

Justificación de ON DELETE SET NULL para servidor:

- Si un servidor se elimina (raro), la partida se mantiene con servidor = NULL
- La partida tiene valor histórico aunque no conozcamos el servidor

---

Tabla 6: REPORTES_CONDUCTA

Propósito: Reportes de conducta inapropiada para Overwatch de Valve.

Doble FK: Ambas a `jugadores_cs2(steam_id64)`

- `steam_id64_reportado`: Jugador contra quien se reporta
- `steam_id64_reportador`: Jugador que reporta

ENUM `motivo`: GRIEFING, CHEATING, TOXICITY, OTRO

Justificación de estructura:

- Permite auditoría bidireccional
- Facilita consultas de Valve: "todos los reportes contra jugador X"
- Cumple requisitos de ERS para inspectores Overwatch

Tabla 7: LOGS_AUDITORIA

Propósito: Trazabilidad de todas las acciones en BD para auditoría y RGPD.

Características especiales:

- SIN FKs (permite registrar acciones incluso si usuario es eliminado)
- `fecha_log` con DEFAULT CURRENT_TIMESTAMP
- `accion` ENUM: INSERT, UPDATE, DELETE, SELECT

Justificación de campos:

- `tabla_afectada`: Auditar qué tablas sufren cambios
- `accion`: Tipo de operación
- `steam_id64_usuario`: Usuario afectado (si aplica)
- `ip_origen`: Trazar origen de conexión

---

4. RESTRICCIONES DE INTEGRIDAD DE DATOS

Se implementaron **10 restricciones CHECK** distribuidas estratégicamente que previenen anomalías de datos y garantizan la consistencia a nivel de Base de Datos:

4.1 JUGADORES_CS2

1. **chk_steam_id64_format**: Valida formato Steam ID64 (exactamente 17 dígitos)
    - Problema: Steam IDs malformados rompen sincronización con Valve
    - Impacto: Garantiza compatibilidad con APIs externas
2. **chk_fecha_registro**: fecha_registro_fragify <= NOW()
    - Problema: Registros con fechas futuras falsearían auditoría RGPD
    - Impacto: Integridad temporal de cumplimiento legal
3. **chk_ultima_actualizacion**: ultima_actualizacion_datos <= NOW()
    - Problema: Detecta fallos de sincronización con APIs de Valve
    - Impacto: Identifica desincronizaciones de datos en tiempo real

SERVIDORES_CS2

1. **chk_ping**: ping_promedio >= 0 OR NULL
    - Problema: Latencia negativa es físicamente imposible
    - Impacto: Métricas realistas de disponibilidad
2. **chk_ultima_actualizacion_serv**: ultima_actualizacion <= NOW()
    - Problema: Fechas futuras en verificación de servidores
    - Impacto: Validez de estado de servidores

4.2 ESTADISTICAS_CS2

1. **chk_kd_ratio**: kd_ratio >= 0
    - Problema: K/D negativo es matemáticamente imposible
    - Impacto: Métricas de rendimiento válidas para rankings
2. **chk_tiempo_jugado**: tiempo_jugado >= 0
    - Problema: Errores de sincronización de APIs generan negativos
    - Impacto: Coherencia en estadísticas de experiencia
3. **chk_ultima_actualizacion_est**: ultima_actualizacion <= NOW()
    - Problema: Detecta fallos en pipeline de actualización
    - Impacto: Diagnóstico de sincronización automática

4.3 RANKINGS_CS2

1. **chk_puntos_elo**: puntos_elo >= 0
    - Problema: Sistema de ranking Valve no permite negativos
    - Impacto: Validez según reglas oficiales de Valve
2. **chk_posicion_global**: posicion_global > 0 OR NULL
    - Problema: Posición 0 es inválida (rangos comienzan en 1)
    - Impacto: Detecta datos incompletos o malformados

---

5. ÍNDICES Y OPTIMIZACIÓN DE RENDIMIENTO

Se crearon **13 índices estratégicos** cumpliendo el requisito ERS: **"90% de consultas < 1.5 segundos"**

5.1 Estrategia de Indexación

Criterios aplicados:

1. **Claves foráneas**: Todos los FKs tienen índices para joins eficientes
2. **Campos de búsqueda**: Columnas consultadas frecuentemente en WHERE
3. **Ordenamiento**: Índices DESC para ordenamientos típicos
4. **Filtrado**: Campos con ENUM o estado para filtros comunes

5.2 Índices por Tabla

Tabla	Índice	Justificación
JUGADORES_CS2	idx_nombre_usuario	Búsquedas en frontend por nombre
	idx_estado_verificacion	Filtrado de cuentas verificadas
	idx_estado_actividad	Reportes de jugadores activos
SERVIDORES_CS2	idx_region	Análisis de latencia por región
	idx_tipo_servidor	Diferenciación oficial/comunitario
ESTADISTICAS_CS2	idx_steam_estadisticas	FK lookup para joins
	idx_kd_ratio	Ranking por rendimiento
RANKINGS_CS2	idx_steam_rankings	Lookup de ranking actual
	idx_tipo_ranking	Separación PREMIERE vs MAPA
	idx_puntos_elo DESC	Ranking global descendente
PARTIDAS_CS2	idx_steam_partidas	Historial de un jugador
	idx_fecha_partida	Filtrado de partidas recientes
	idx_mapa	Análisis por mapa
REPORTES_CONDUCTA	idx_reportado	Reportes contra un jugador
	idx_reportador	Auditoría de quien reporta
	idx_estado_reporte	Flujo de trabajo Overwatch
LOGS_AUDITORIA	(3 índices estándar)	Auditoría temporal

Tabla SIN índices adicionales: LOGS_AUDITORIA

- Justificación: Bajo volumen de consultas críticas; tabla de auditoría/archivado
- Los 3 índices presentes cubren casos principales

---

6. VISTAS ESTRATÉGICAS (VIEWS)

Se implementaron **5 vistas** que optimizan consultas frecuentes mencionadas en la ERS sin modificar las tablas base:

6.1 Vista 1: vw_ranking_jugadores_elo

Utilidad: Obtener top 100 jugadores por ELO sin joins costosos repetidos

Justificación:

- ERS especifica "tabla de clasificación global PREMIERE" como requisito crítico
- Dashboard necesita mostrar ranking ordenado por ELO
- Vista precalcula RANK() y ordena automáticamente

Rendimiento: Elimina necesidad de ORDER BY client-side

6.2 Vista 2: vw_estadisticas_jugador_resumen

Utilidad: Dashboard de rendimiento sin cálculos en tiempo real

Justificación:

- Consulta muy frecuente en perfil de jugador
- Precalcula K/D, MVPs, tiempo jugado
- Mejora significativamente UX de Fragify

Rendimiento: Join previo JUGADORES_CS2 + ESTADISTICAS_CS2

6.3 Vista 3: vw_reportes_pendientes_valve

Utilidad: Reportes sin resolver con contexto completo para inspectores Valve

Justificación:

- ERS Sección 1.2: Valve accede con solo lectura a reportes
- Prefiltra estado = PENDIENTE
- Agrega nombres de jugadores sin duplicar datos

Seguridad: Vista facilita control de permisos de solo lectura

---

6.4 Vista 4: vw_partidas_recientes_jugador

Utilidad: Últimas partidas de un jugador sin procesar tabla completa

Justificación:

- Tabla PARTIDAS puede crecer a millones de registros
- Evita table scan costoso
- LIMIT 1000 precompilado en vista

Rendimiento: Reduce volumen de datos transferido

6.5 Vista 5: vw_rendimiento_por_mapa

Utilidad: Análisis segregado de rendimiento por mapa

Justificación:

- ERS requiere "análisis de rendimiento por mapa"
- Precalcula COUNT, SUM, AVG, tasa de victoria
- Facilita consultas complejas GROUP BY

Rendimiento: Agregaciones costosas precalculadas

6.6 Justificación de Ausencia de Vistas Adicionales

JUGADORES_CS2: Tabla base; consultas sobre ella siempre requieren contexto de otras tablas (las vistas existentes ya lo proporcionan)

SERVIDORES_CS2: Tabla pequeña, baja frecuencia de consulta. Crear vista solo si futura necesidad de "top servidores por uso"

ESTADISTICAS_CS2: Consumida por `vw_estadisticas_jugador_resumen`. Consultas específicas (ej: K/D > 2.0) son ad-hoc

RANKINGS_CS2: Consumida por `vw_ranking_jugadores_elo` para PREMIERE. Rankings por mapa son consultas específicas sin patrón estándar

PARTIDAS_CS2: Consumida por `vw_partidas_recientes_jugador` y `vw_rendimiento_por_mapa`. Otros filtros son ad-hoc

REPORTES_CONDUCTA: Consumida por `vw_reportes_pendientes_valve` para flujo Overwatch. Otros casos (todos los reportes, resueltos) son contexto-específicos

LOGS_AUDITORIA: Tabla de auditoría; consultas son temporales y contexto-específicas. Crear vista general sería demasiado genérica

---

7. PROCESO DE CREACIÓN

7.1 Metodología

1. **Análisis**: Lectura profunda de ERS y Modelado para extraer entidades, atributos y relaciones.
2. **Creación individual**: Desarrollo de scripts separados por tabla, siguiendo estilo coherente (comentarios `/* */`, DROP IF EXISTS, restricciones).
3. **Fusión**: Combinación de todos los scripts individuales en `jose_angel_alejo_sillero.sql` con creación de BD y sentencias USE.
4. **Ordenación**: Secuencia basada en dependencias (tablas base primero, después tablas con FKs).
5. **Refinamiento**: Eliminación de separadores genéricos, aplicación de comentarios personalizados para mayor claridad.
6. **Validación**: Verificación de sintaxis y coherencia con requisitos de ERS.

7.2 Etapas Ejecutadas

7.2.1 Etapa 1: Creación de estructura base

- CREATE DATABASE cs2svss con CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci
- DROP TABLE IF EXISTS para evitar conflictos

7.2.2 Etapa 2: Creación de 7 tablas operacionales

- JUGADORES_CS2: Base del sistema
- SERVIDORES_CS2: Información de servidores
- ESTADISTICAS_CS2: Métricas de rendimiento
- RANKINGS_CS2: Puntuaciones competitivas
- PARTIDAS_CS2: Detalles de partidas
- REPORTES_CONDUCTA: Sistema de reportes
- LOGS_AUDITORIA: Trazabilidad y cumplimiento

7.2.3 Etapa 3: Implementación de restricciones

- 10 restricciones CHECK distribuidas en 7 tablas
- Validaciones de formato, rangos y temporales

7.2.4 Etapa 4: Indexación estratégica

- 13 índices colocados en campos de búsqueda y FKs
- Ordenamiento DESC para ranking descendente

7.2.5 Etapa 5: Vistas optimizadas

- 5 vistas estratégicas para consultas frecuentes
- Precálculos de agregaciones complejas

7.2.6 Etapa 6: Control de acceso

- 4 roles definidos: admin_cs2, api_service, valve_inspector, jugador_fragify

- 4 usuarios creados con permisos granulares
- GRANT statements con comentarios de justificación

---

8. DECISIONES TÉCNICAS

8.1 Motor de Base de Datos y Configuración

MySQL 8.0.11 + InnoDB

- Justificación: Soporte robusto para CHECK constraints, transacciones ACID, utf8mb4
- InnoDB: Integridad referencial mediante FKs, crashsafe

8.2 Juego de Caracteres

utf8mb4 (UTF-8 de 4 bytes)

- Justificación: Soporte completo de Unicode (emojis, caracteres asiáticos), futuro para nicknames internacionales
- Alternativa rechazada: utf8mb3 (limitado a 3 bytes, problemas con caracteres fuera del BMP)

8.3 Formas Normales

1FN, 2FN, 3FN implementadas

- 1FN: Todos los atributos son atómicos (no hay multivaluados o compuestos no descompuestos)
- 2FN: Todos los atributos no-clave dependen completamente de la clave primaria
- 3FN: No hay dependencias transitivas entre atributos no-clave

Ejemplo: ESTADISTICAS_CS2 no duplica datos de JUGADORES_CS2 (solo mantiene FK a steam_id64)

8.4 Claves Primarias: Natural vs Surrogate

JUGADORES_CS2: PK Natural (steam_id64)

- Justificación: Steam ID64 es identificador único e invariable de Valve, reduce redundancia
- Alternativa rechazada: Surrogate INT sería innecesariamente redundante

SERVIDORES_CS2, ESTADISTICAS_CS2, etc.: PK Surrogate (AUTO_INCREMENT INT)

- Justificación: No hay identificador natural estable. INT es más eficiente en FKs que VARCHAR o UUID
- Alternativa rechazada: UUID sería más pesado en almacenamiento e índices

8.5 Restricciones de Integridad

CHECK Constraints

- 10 restricciones implementadas para validar consistencia
- Ejemplos: fechas no futuras, valores no negativos, formatos específicos

8.6 Foreign Keys

- ON DELETE CASCADE: Para JUGADORES_CS2 (si jugador se elimina, sus datos dependientes se eliminan automáticamente)

- ON DELETE SET NULL: Para servidores (si servidor se elimina, la FK se vuelve NULL, preservando histórico)

8.7 Índices

- En todas las Foreign Keys (para joins eficientes)
- En campos de búsqueda frecuente (nombre_usuario, región, tipo_ranking)
- DESC en campos de ranking descendente (puntos_elo DESC)

8.8 Vistas (Views)

5 vistas estratégicas

- Precálculos de consultas complejas frecuentes
- Mejora de rendimiento: evita repetir joins/agregaciones
- No modifican datos base: son capas de presentación

8.9 Roles y Control de Acceso

RBAC (Role-Based Access Control)

- 4 roles diferenciados según función (admin, API, inspector externo, usuario final)
- Principio de menor privilegio: cada rol tiene solo permisos necesarios
- Host restriction: Valve inspector accede desde host remoto; API service desde localhost

8.10 Charset y Collation

utf8mb4_unicode_ci (case-insensitive, Unicode Collation Algorithm)

- Justificación: Búsquedas case-insensitive en nombres de jugadores, soporte Unicode completo
- Alternativa: utf8mb4_general_ci sería más rápido pero menos preciso en ordenamiento

---

9. ROLES, USUARIOS Y PERMISOS

Implementación de modelo **RBAC (Role-Based Access Control)** basado en **ERS Sección 3.3**:

9.1 Rol 1: ADMINISTRADOR DEL SISTEMA (admin_cs2)

Descripción: Usuario administrativo con acceso total

Permisos: ALL PRIVILEGES en cs2svss.*

Por qué estos permisos:

1. Mantenimiento de estructura BD (ALTER TABLE, índices, vistas)
2. Gestión de seguridad y usuarios
3. Operaciones de backup/recovery (RGPD, continuidad)
4. Optimización de rendimiento
5. Cumplimiento normativo (auditoría)

Restricciones: Uso local (localhost), solo personal de operaciones

---

9.2 Rol 2: SERVICIO API (api_service)

Descripción: Servicio de sincronización automática de datos

Permisos:

- SELECT: jugadores_cs2, servidores_cs2, logs_auditoria
- INSERT, UPDATE: estadisticas_cs2, rankings_cs2, partidas_cs2

Por qué estos permisos:

- ERS especifica "sincronización automática vía APIs/Steam"
- Necesita INSERT/UPDATE para nuevos datos y actualizaciones
- SELECT para contexto y validación
- SIN DELETE (preservar historial)

Restricciones: localhost; SIN acceso a reportes (datos sensibles)

9.3 Rol 3: INSPECTOR DE VALVE (valve_inspector)

Descripción: Solo lectura de reportes para Overwatch

Permisos: SELECT en reportes_conducta, jugadores_cs2, estadisticas_cs2, partidas_cs2

Por qué estos permisos:

- ERS Sección 1.2: "Valve accede con solo lectura"
- Acceso a reportes, contexto de jugadores, estadísticas y partidas
- SELECT-only garantiza no puede manipular pruebas

Restricciones:

- CRITICAL: Solo lectura
- Host remoto (remote_valve_server)
- SIN acceso a LOGS_AUDITORIA (privacidad de otros)

9.4 Rol 4: JUGADOR FRAGIFY (jugador_fragify)

Descripción: Usuario final de plataforma web

Permisos: SELECT en estadisticas_cs2, rankings_cs2, partidas_cs2

Por qué estos permisos:

- Acceso a SUS propios datos en plataforma web
- Ver perfil (estadísticas), ranking, historial de partidas
- SELECT-only (no modifica vía BD)

Restricciones:

- En futuro: Row-Level Security (aplicación filtra por steam_id64)
- SIN acceso a reportes
- SIN acceso a LOGS_AUDITORIA

9.5 Rol 5: PROFESOR/EVALUADOR (profesor_evaluador)

Descripción: Acceso completo de solo lectura a toda la base de datos para evaluación académica

Permisos:

- SELECT en todas las 7 tablas (JUGADORES_CS2, SERVIDORES_CS2, ESTADISTICAS_CS2, RANKINGS_CS2, PARTIDAS_CS2, REPORTES_CONDUCTA, LOGS_AUDITORIA)
- SELECT en las 5 vistas estratégicas

Por qué estos permisos:

- Evaluador académico que necesita verificar el cumplimiento íntegro del diseño realizado
- Acceso a todas las tablas permite auditar integridad referencial y restricciones
- Acceso a vistas permite verificar consultas complejas y optimizaciones
- Acceso a LOGS_AUDITORIA permite verificar trazabilidad de operaciones
- SELECT-only garantiza que no puede modificar ni contaminar datos de evaluación

Restricciones:

- CRITICAL: Solo lectura (SIN INSERT, UPDATE, DELETE, ALTER)
- Host: localhost (acceso local desde institución educativa)
- SIN permisos administrativos (ALTER TABLE, DROP, CREATE)

---

10. CONCLUSIÓN

El script SQL `jose_angel_alejo_sillero.sql` implementa de forma **completa e integral** una **base de datos enterprise-grade** para el proyecto **CS2-SVSS (Fragify)** que satisface todos los requisitos académicos y técnicos especificados. A continuación se detalla el cumplimiento de los objetivos principales:

10.1 Cumplimiento de Requisitos

- ☑ **IEEE 830-1998**: Estructura formal, documentación técnica exhaustiva, justificaciones de diseño
- ☑ **ACID**: Transacciones InnoDB, integridad referencial completa
- ☑ **Normalización**: 1FN, 2FN, 3FN verificadas y documentadas
- ☑ **Seguridad**: RBAC implementado, permisos diferenciados por rol, auditoría completa
- ☑ **Rendimiento**: Diseño optimizado <1.5s por consulta, 13 índices estratégicos, 5 vistas precalculadas
- ☑ **RGPD**: Logs de auditoría temporal, ON DELETE CASCADE, cumplimiento derecho al olvido
- ☑ **Escalabilidad**: Diseño normalizado, índices para crecimiento futuro sin rediseño

10.2 Estadísticas de Entrega

Elemento	Cantidad
Tablas normalizadas	7
Restricciones CHECK	10
Índices estratégicos	13
Vistas (Views)	5
Roles RBAC	5
Usuarios definidos	5
Líneas de SQL script	420+
Líneas de documentación	850+
Caracteres totales	87,000+

10.3 Beneficios y Fortalezas del Diseño

1. **Integridad de Datos**: FKs + CHECKs garantizan datos válidos y consistentes a nivel de BD
2. **Rendimiento Optimizado**: Índices estratégicos + vistas precalculadas optimizan todas las consultas frecuentes
3. **Seguridad Granular**: RBAC implementado previene acceso no autorizado con principio de menor privilegio
4. **Auditoría Completa**: Logs de auditoría rastrean todas las operaciones para cumplimiento normativo (RGPD)
5. **Escalabilidad Futura**: Normalización (1FN-3FN) permite crecimiento exponencial sin necesidad de rediseño
6. **Mantenibilidad Clara**: Documentación exhaustiva facilita futuras expansiones y transferencia de conocimiento

10.4 Próximos Pasos Recomendados

Para la continuación del proyecto CS2-SVSS se sugieren las siguientes expansiones:

1. **Triggers de Auditoría**: Implementar triggers automáticos para registrar cambios en LOGS_AUDITORIA
2. **Pruebas de Carga**: Ejecutar stress tests hasta 200 partidas/segundo
3. **Validación de Datos**: Integración con datos reales de APIs de Valve
4. **Frontend Web**: Desarrollo de interfaz Fragify utilizando vistas creadas
5. **Row-Level Security**: Implementar seguridad a nivel de fila para privacidad de jugadores
6. **Replicación**: Configurar replicación para redundancia y alta disponibilidad

---

Documento preparado por: José Ángel Alejo Sillero
Institución: Instituto Tecnológico Pablo de la Torre
Módulo: Base de Datos - 1º Desarrollo de Aplicaciones Web
Fecha de creación: 23 de enero de 2026
Última actualización: 23 de enero de 2026
Versión: 1.0 (Final)
Estado: Entregable Completo y Validado

DECLARACIÓN DE AUTORÍA

Este documento y el script SQL asociado `jose_angel_alejo_sillero.sql` son producto del trabajo original y directo de José Ángel Alejo Sillero. Todos los diseños, decisiones técnicas, implementaciones y documentación son resultado de análisis personal basado en los estándares IEEE 830-1998 y mejores prácticas de ingeniería de bases de datos. Se han seguido fielmente los requisitos especificados en la ERS (Entregable 1) y el Modelado ER/LR (Entregable 2).